Name: Aditi Mittal
PRN: 23610067
Class: L.Y. CSE
Batch: B1

Experiment no 6

Assignment 6: Introduction to MPI : Installation, Processes, Ranks, Groups and Point-to-Point Communication

Aim:
To configure the MPI environment and understand the fundamentals of distributed-memory parallel programming by creating MPI processes, identifying process ranks, working with MPI groups, and implementing one-to-one communication using MPI_Send() and MPI_Recv().

Why Are We Learning This?
OpenMP allows multiple threads to work in a shared memory space. However, many HPC applications run on clusters where each computing node has its own memory.

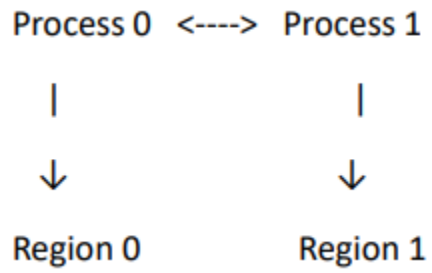
MPI (Message Passing Interface) provides a standard programming model for such distributed memory systems.

In MPI:
- A program is executed by multiple processes.
- Each process has its own address space.
- Processes communicate by exchanging messages.
- Each process is identified by a unique rank.
- Processes can be organized into groups and communicators.

MPI is widely used in:
- Weather and climate simulation
- Computational fluid dynamics
- Molecular dynamics
- Large-scale scientific simulations
- Distributed AI/ML workloads
- Big-data processing
- Supercomputing applications

HPC in Practice
Consider a weather simulation running on 100 computing nodes.
Each process may calculate the weather conditions for a particular geographical region. However, neighbouring regions need to exchange information.

For example:



MPI provides the communication mechanism required to exchange this information.

If each process has its own memory:
How can Process 0 send data to Process 1?
The answer is message passing.

This assignment introduces the basic mechanism using:
MPI_Send()
MPI_Recv()

Learning Focus
This assignment introduces:
• MPI environment
• MPI program structure
• MPI initialization and termination
• MPI processes
• MPI rank
• MPI size
• MPI communicator
• MPI groups
• Point-to-point communication
• MPI_Send()
• MPI_Recv()
• Message tags
• Source and destination
• Basic performance measurement

Pre-Lab Activity
Answer the following questions before starting the assignment.

1. What is distributed-memory parallel computing?

Distributed-memory parallel computing is a type of parallel computing in which multiple processors or computers work together, but **each processor has its own separate memory**.

Since one processor cannot directly access another processor's memory, they communicate by **sending and receiving messages**.

Example:
If 4 computers are working on a large problem, each computer gets its own portion of the data and processes it independently. They exchange information whenever required.

2. What is MPI?

MPI (Message Passing Interface) is a **standard for communication between processes** in parallel and distributed-memory programs.

It provides functions that allow processes to:

- Send data to other processes
- Receive data from other processes
- Synchronize with each other
- Perform collective operations such as broadcast and reduction

MPI is commonly used with languages such as **C, C++, and Fortran**.

3. What is the difference between a process and a thread?

Process	Thread
An independent program execution unit	A smaller execution unit within a process
Has its **own memory space**	Shares memory with other threads of the same process
Communication between processes usually requires mechanisms such as message passing	Threads can communicate through shared memory
More expensive to create and manage	Generally lighter and faster to create
MPI primarily works with **processes**	OpenMP commonly works with **threads**

Simple example:
If a program is running as 4 MPI processes, each process has its own memory. If one process creates 4 threads, those threads share the memory of that process.

4. What is an MPI rank?

An **MPI rank** is a unique integer identifier assigned to each process within an MPI communicator.

Ranks normally start from **0**.

For example, if an MPI program runs using 4 processes:

Process 0 → Rank 0
Process 1 → Rank 1
Process 2 → Rank 2
Process 3 → Rank 3

The rank allows a process to identify **itself** and specify **which process should receive a message**.

5. What is an MPI communicator?

An **MPI communicator** is an object that defines a **group of processes that can communicate with each other**.

It provides the communication context in which MPI operations take place.

For example, if a communicator contains 4 processes, those processes can communicate using their ranks within that communicator.

A communicator therefore tells MPI:

> **Which processes belong to the communication group and how the communication is organized.**

6. What is the purpose of MPI_COMM_WORLD?

`MPI_COMM_WORLD` is the **default communicator created by MPI**.

It contains **all the processes participating in the MPI program**.

For example, if we start a program with:

mpirun -np 4 ./program

then `MPI_COMM_WORLD` contains all 4 processes:

MPI_COMM_WORLD
    |
    ├── Rank 0

├── Rank 1
├── Rank 2
└── Rank 3

It is commonly used to determine:

- The total number of processes using `MPI_Comm_size()`
- The rank of the current process using `MPI_Comm_rank()`
- Communication between processes using functions such as `MPI_Send()` and `MPI_Recv()`

7. What is the difference between MPI_Send() and MPI_Recv()?

`MPI_Send()` and `MPI_Recv()` are used for **point-to-point communication** between MPI processes.

`MPI_Send()`	`MPI_Recv()`
Sends data from one process to another	Receives data from another process
Used by the **sender**	Used by the **receiver**
Specifies the destination process	Specifies the source process
Transfers data to another process	Obtains data sent by another process

Example:

MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);

This sends `data` from the current process to **rank 1**.

The receiving process can use:

MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);

This receives data from **rank 0**.

Process 0 Process 1
| |
| -------- MPI_Send() ----------> |
| |
| <--------- MPI_Recv() --------- |

8. Why is message passing necessary in distributed-memory systems?

Message passing is necessary because **each process has its own separate memory**.

A process cannot directly access or modify the memory of another process. Therefore, when processes need to exchange data or coordinate their work, they must communicate by **sending messages**.

For example:

Process 0 Process 1
Memory A Memory B
| |
| ---- send data ----------> |
| |
| <--- send result ---------- |

MPI provides the mechanisms needed for this communication.

In short:

> In distributed-memory computing, message passing allows independent processes with separate memories to exchange data and coordinate their execution.

Section A – Basics

Task 1: MPI Installation and Environment Verification

Install and configure an MPI implementation such as OpenMPI or MPICH.
Verify the installation using appropriate commands.

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC$ sudo apt install mpich
Installing:
  mpich

Installing dependencies:
  autoconf                      libgfortran-15-dev  libibumad3    libucx-dev
  automake                      libgfortran5        libltdl-dev   libucx0
  autotools-dev                 libhsa-runtime64-1  libltdl7      libxnvctrl0
  gfortran-15                   libhsakmt1          libmpich-dev  libze1
  gfortran-15-x86-64-linux-gnu  libhwloc-dev        libmpich12    m4
  hwloc-nox                     libhwloc-plugins    libnuma-dev   ocl-icd-libopencl1
  libamd-comgr3                 libhwloc15          libnuma1
  libamdhip64-7                 libibmad5           libtool
```

mpicc --version

and:

mpirun --version

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC$ mpicc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

or:

mpiexec --version

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC$ mpiexec --version
HYDRA build details:
    Version:                                 4.3.2
    Release Date:                            Mon Oct  6 11:14:20 AM CDT 2025
    CC:                              gcc -Wdate-time -D_FORTIFY_SOURCE=3 -g -O2 -Werror=i
mplicit-function-declaration -fno-omit-frame-pointer -mno-omit-leaf-frame-pointer -flto=a
uto -ffat-lto-objects -fstack-protector-strong -fstack-clash-protection -Wformat -Werror=
format-security -fcf-protection -fdebug-prefix-map=/build/mpich-f8z5Zh/mpich-4.3.2=/usr/s
rc/mpich-4.3.2-2 -Wl,-Bsymbolic-functions -flto=auto -ffat-lto-objects -Wl,-z,relro -Wl,-
z,now
```

Environment Details

Parameter	Details
MPI Implementation	**MPICH**
MPI Version	**4.3.2**
C Compiler	**15.2.0**
Operating System	**Linux**
Processor/CPU	**13th Gen Intel(R) Core(TM) i9-13950HX**
Number of CPU Cores	**32**

Verification

The successful execution of `mpicc --version` and `mpirun --version` confirmed that the MPI compiler and runtime environment were available.

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC$ mpichversion
MPICH Version:      4.3.2
MPICH Release date: Mon Oct  6 11:14:20 AM CDT 2025
MPICH ABI:          17:2:5
MPICH Device:       ch4:ucx
MPICH configure:    --build=x86_64-linux-gnu --prefix=/usr --includedir=${prefix}/i
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC$ lscpu
Architecture:                x86_64
  CPU op-mode(s):            32-bit, 64-bit
  Address sizes:             46 bits physical, 48 bits virtual
  Byte Order:                Little Endian
CPU(s):                      32
  On-line CPU(s) list:       0-31
Vendor ID:                   GenuineIntel
  Model name:                13th Gen Intel(R) Core(TM) i9-13950HX
    CPU family:              6
    Model:                   183
    Thread(s) per core:      2
    Core(s) per socket:      16
    Socket(s):               1
    Stepping:                1
```

Task 2: First MPI Program

Write an MPI program to display:
Hello from MPI
The program must contain the appropriate MPI initialization and termination routines.

Identify the purpose of:
MPI_Init()
MPI_Finalize()

Compile and execute the program using:
mpicc hello.c -o hello
mpirun -np 4 ./hello

Objective

To create a basic MPI program that initializes the MPI environment, prints a message, and terminates the MPI environment.

Program: `hello.c`

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    printf("Hello from MPI\n");

    MPI_Finalize();

    return 0;
}
```

Compilation

mpicc hello.c -o hello

Execution

mpirun -np 4 ./hello

Output

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC$ cd A6/Task2
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/Task2$ mpicc hello.c -o hel
lo
mpirun -np 4 ./hello
Hello from MPI
Hello from MPI
Hello from MPI
Hello from MPI
```

The message is printed once by each of the four MPI processes.

Purpose of MPI_Init()

`MPI_Init()` initializes the MPI execution environment. It must be called before most MPI functions can be used.

MPI_Init(&argc, &argv);

It prepares the MPI processes for communication and establishes the MPI environment.

Purpose of MPI_Finalize()

`MPI_Finalize()` terminates the MPI environment after all required MPI operations have been completed.

MPI_Finalize();

It should normally be called before the program exits.

Task 3: Understanding MPI Rank and Size

Modify the program to display:
• Process rank
• Total number of processes

Use:
MPI_Comm_rank()
MPI_Comm_size()

Execute using:
• 2 processes
• 4 processes
• 8 processes

Expected Output
The order of output may vary.
Process 0 of 4
Process 1 of 4
Process 2 of 4
Process 3 of 4

Record Your Observation
Number of Processes Observed Ranks Total Processes
2
4
8

Program: `rank_size.c`

```
#include <stdio.h>
```

```
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank, size;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   printf("Process %d of %d\n", rank, size);

   MPI_Finalize();

   return 0;
}
```

Compilation

mpicc rank_size.c -o rank_size

Execution with 2 Processes

mpirun -np 2 ./rank_size

Expected Output

Process 0 of 2
Process 1 of 2

Execution with 4 Processes

mpirun -np 4 ./rank_size

Expected Output

Process 0 of 4
Process 1 of 4
Process 2 of 4
Process 3 of 4

Execution with 8 Processes

mpirun -np 8 ./rank_size

Expected Output

Process 0 of 8
Process 1 of 8
Process 2 of 8
Process 3 of 8
Process 4 of 8
Process 5 of 8
Process 6 of 8
Process 7 of 8

The order of output may vary because the MPI processes execute concurrently.

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/Task3$ mpicc rank_size.c -o
 rank_size
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/Task3$ mpirun -np 2 ./rank_
size
Process 0 of 2
Process 1 of 2
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/Task3$ mpirun -np 4 ./rank_
size
Process 0 of 4
Process 1 of 4
Process 2 of 4
Process 3 of 4
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/Task3$ mpirun -np 8 ./rank_
size
Process 0 of 8
Process 4 of 8
Process 5 of 8
Process 6 of 8
Process 1 of 8
Process 2 of 8
Process 3 of 8
Process 7 of 8
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/Task3$
```

Observation

Number of Processes	Observed Ranks	Total Processes
2	0, 1	2
4	0, 1, 2, 3	4
8	0, 1, 2, 3, 4, 5, 6, 7	8

Conclusion

`MPI_Comm_rank()` returns the rank of the current process, while `MPI_Comm_size()` returns the total number of processes in the communicator. The ranks start from 0 and continue up to `size - 1`.

Task 4: Understanding MPI Groups and Communicators

Investigate the relationship between:
MPI_COMM_WORLD → MPI Group → MPI Communicator
Using MPI group-related functions, create a subgroup containing selected processes from MPI_COMM_WORLD.
For example, if there are 8 processes:
MPI_COMM_WORLD
Rank: 0 1 2 3 4 5 6 7
↓ ↓ ↓ ↓
Selected: 0 2 4 6

Create a subgroup containing selected ranks and explore:
• MPI_Comm_group()
• MPI_Group_incl()
• MPI_Comm_create_group()

Display the rank of each process in the newly created communicator.
Important
Understand the difference between:
Global rank
and
Rank within a communicator.

Think Like an HPC Programmer
Consider 16 processes running on an HPC cluster.
Suppose:
• Processes 0–7 perform one computational task.
• Processes 8–15 perform another computational task.

Objective

To create a subgroup containing selected processes from `MPI_COMM_WORLD` and create a new communicator for those processes.

For this experiment, assume that the program is executed using **8 processes** and the selected global ranks are:

0, 2, 4, 6

Program: `group_comm.c`

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int world_rank, world_size;
   int new_rank, new_size;

   MPI_Group world_group;
   MPI_Group new_group;
   MPI_Comm new_comm;

   int ranks[] = {0, 2, 4, 6};

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
   MPI_Comm_size(MPI_COMM_WORLD, &world_size);

   MPI_Comm_group(MPI_COMM_WORLD, &world_group);

   MPI_Group_incl(world_group, 4, ranks, &new_group);

   MPI_Comm_create_group(
      MPI_COMM_WORLD,
      new_group,
      0,
      &new_comm
   );

   if (new_comm != MPI_COMM_NULL)
   {
      MPI_Comm_rank(new_comm, &new_rank);
      MPI_Comm_size(new_comm, &new_size);

      printf(
         "Global Rank = %d, New Communicator Rank = %d, New Size = %d\n",
         world_rank,
         new_rank,
         new_size
```

```
    );

    MPI_Comm_free(&new_comm);
  }

  MPI_Group_free(&new_group);
  MPI_Group_free(&world_group);

  MPI_Finalize();

  return 0;
}
```

Compilation

mpicc group_comm.c -o group_comm

Execution

mpirun -np 8 ./group_comm

Observation

Only the selected processes participate in the new communicator:

Global Rank	New Communicator Rank
0	0
2	1
4	2
6	3

Processes with global ranks **1, 3, 5, and 7** do not belong to the new communicator.

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/task4$ mpicc group_comm.c -
o group_comm
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/task4$ mpirun -np 8 ./group
_comm
Global Rank = 0, New Communicator Rank = 0, New Size = 4
Global Rank = 2, New Communicator Rank = 1, New Size = 4
Global Rank = 4, New Communicator Rank = 2, New Size = 4
Global Rank = 6, New Communicator Rank = 3, New Size = 4
```

Important Observation

The global rank and communicator rank are not necessarily the same.

For example:

Global Rank 4 → New Communicator Rank 2
Global Rank 6 → New Communicator Rank 3

This demonstrates that an MPI rank is always interpreted **relative to a particular communicator**.

Answer:
1. Why might it be useful to create separate communicators?

Separate communicators are useful when we want **different groups of MPI processes to communicate independently**.

For example, in a large application, some processes may be responsible for image processing while others handle data analysis. Separate communicators can be created for these groups so that communication within one group does not interfere with communication in another group.

This helps:

- Organize communication
- Improve program structure
- Avoid unintended communication between processes
- Allow different parts of an application to work independently

2. Can the same process belong to different communication groups?

Yes. The same MPI process can belong to multiple groups and communicators.

For example, a process might belong to:

```
MPI_COMM_WORLD
    |
    ├── Group A → Data Processing
    |
    └── Group B → Analysis
```

The process can have a **different rank in each communicator** because MPI ranks are assigned relative to the communicator.

Therefore, a process could be **rank 2 in one communicator and rank 0 in another**.

3. Why is the communicator important in MPI communication?

A communicator defines the **communication context and the group of processes** involved in an MPI operation.

When using functions such as:

MPI_Send(..., destination, ..., communicator);
MPI_Recv(..., source, ..., communicator, ...);

the communicator determines which group of processes the ranks belong to and provides a separate communication context.

This is important because the same rank number can exist in different communicators without causing confusion.

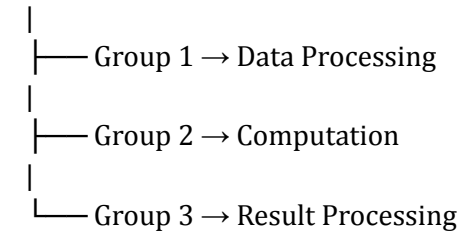
In simple terms:

> **The communicator tells MPI which group of processes the communication belongs to and provides the context in which their ranks are interpreted.**

4. How can groups help organize large parallel applications?

Groups help organize a large MPI application by allowing processes to be **logically divided according to their tasks**.

For example:

All MPI Processes


Each group can contain the processes responsible for a particular part of the application. Communicators can then be created for these groups to allow communication within them.

This makes a large parallel application easier to **design, manage, and coordinate**, while also reducing unnecessary communication between unrelated processes.

Section B – Application: One-to-One Communication

Task 5: MPI Send and Receive

Implement a program in which Process 0 sends a message to Process 1.
Process 1 should receive the message and display it.
Use:
MPI_Send()
MPI_Recv()

Objective

To send a message from Process 0 to Process 1 using `MPI_Send()` and `MPI_Recv()`.

Program: `send_receive.c`

```
#include <stdio.h>
#include <string.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank;
   char message[100];

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);

   if (rank == 0)
   {
      strcpy(message, "Hello from Process 0");

      MPI_Send(
         message,
         strlen(message) + 1,
         MPI_CHAR,
         1,
         0,
         MPI_COMM_WORLD
      );

      printf("Process 0 sent: %s\n", message);
```

```
    }
    else if (rank == 1)
    {
        MPI_Recv(
            message,
            100,
            MPI_CHAR,
            0,
            0,
            MPI_COMM_WORLD,
            MPI_STATUS_IGNORE
        );

        printf("Process 1 received: %s\n", message);
    }

    MPI_Finalize();

    return 0;
}
```

Compilation

mpicc send_receive.c -o send_receive

Execution

mpirun -np 2 ./send_receive

Expected Output

Process 0 sent: Hello from Process 0
Process 1 received: Hello from Process 0

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/task5$ mpicc send_receive.c
 -o send_receive
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/task5$ mpirun -np 2 ./send_
receive
Process 0 sent: Hello from Process 0
Process 1 received: Hello from Process 0
```

Observation

Process 0 acts as the sender and Process 1 acts as the receiver. The message is transferred using point-to-point communication.

Task 6: Integer Data Communication
Modify the previous program so that Process 0 sends an integer value to Process 1.
Example:
Process 0:
Value = 100
MPI_Send()
↓
Process 1:
Received Value = 100
Verify the received value.

Objective

To send an integer value from Process 0 to Process 1.

Program: `integer_send.c`

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank;
   int value;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);

   if (rank == 0)
   {
      value = 100;

      MPI_Send(
         &value,
         1,
         MPI_INT,
         1,
         0,
         MPI_COMM_WORLD
      );
```

```
    printf("Process 0: Value = %d\n", value);
  }
  else if (rank == 1)
  {
    MPI_Recv(
      &value,
      1,
      MPI_INT,
      0,
      0,
      MPI_COMM_WORLD,
      MPI_STATUS_IGNORE
    );

    printf("Process 1: Received Value = %d\n", value);
  }

  MPI_Finalize();

  return 0;
}
```

Compilation

mpicc integer_send.c -o integer_send

Execution

mpirun -np 2 ./integer_send

Expected Output

Process 0: Value = 100
Process 1: Received Value = 100

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/task6$ mpicc integer_send.c
 -o integer_send
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/task6$ mpirun -np 2 ./integ
er_send
Process 0: Value = 100
Process 1: Received Value = 100
```

Observation

The integer value sent by Process 0 was correctly received by Process 1, demonstrating communication of numerical data using `MPI_Send()` and `MPI_Recv()`.

Task 7: Array Communication

Extend the program so that Process 0 sends an array of integers to Process 1.
For example:
Process 0
10 20 30 40 50
|
| MPI_Send()
↓
Process 1
10 20 30 40 50
Process 1 should calculate the:
• Sum
• Average
• Maximum
of the received array.

Objective

To send an array of integers from Process 0 to Process 1 and calculate its sum, average, and maximum.

Program: `array_send.c`

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank;
   int data[5] = {10, 20, 30, 40, 50};
   int received[5];

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);

   if (rank == 0)
```

```
{
    MPI_Send(
        data,
        5,
        MPI_INT,
        1,
        0,
        MPI_COMM_WORLD
    );

    printf("Process 0 sent array: ");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", data[i]);
    }

    printf("\n");
}
else if (rank == 1)
{
    MPI_Recv(
        received,
        5,
        MPI_INT,
        0,
        0,
        MPI_COMM_WORLD,
        MPI_STATUS_IGNORE
    );

    int sum = 0;
    int max = received[0];

    for (int i = 0; i < 5; i++)
    {
        sum += received[i];

        if (received[i] > max)
        {
            max = received[i];
        }
```

```
    }

    double average = (double)sum / 5;

    printf("Process 1 received array: ");

    for (int i = 0; i < 5; i++)
    {
       printf("%d ", received[i]);
    }

    printf("\n");
    printf("Sum = %d\n", sum);
    printf("Average = %.2f\n", average);
    printf("Maximum = %d\n", max);
  }

  MPI_Finalize();

  return 0;
}
```

Compilation

mpicc array_send.c -o array_send

Execution

mpirun -np 2 ./array_send

Expected Output

Process 0 sent array: 10 20 30 40 50
Process 1 received array: 10 20 30 40 50
Sum = 150
Average = 30.00
Maximum = 50

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/task7$ mpicc array_send.c -
o array_send
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/task7$ mpirun -np 2 ./array
_send
Process 0 sent array: 10 20 30 40 50
Process 1 received array: 10 20 30 40 50
Sum = 150
Average = 30.00
Maximum = 50
```

Observation

The complete array was successfully transferred from Process 0 to Process 1. Process 1 then performed the required calculations.

Performance Activity
Measure the communication time for different message sizes.
Test with:
• 1 KB
• 10 KB
• 100 KB
• 1 MB
• 10 MB

Record:
Message Size Communication Time
1 KB
Message Size Communication Time
10 KB
100 KB
1 MB
10 MB

The communication performance can be measured by sending messages of different sizes between two MPI processes.

Program: `communication_time.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank;
   int sizes[] = {
      1024,
      10 * 1024,
      100 * 1024,
      1024 * 1024,
      10 * 1024 * 1024
   };
```

```
int num_sizes = 5;

MPI_Init(&argc, &argv);

MPI_Comm_rank(MPI_COMM_WORLD, &rank);

for (int s = 0; s < num_sizes; s++)
{
   int bytes = sizes[s];
   char *buffer = (char *)malloc(bytes);

   if (rank == 0)
   {
      double start = MPI_Wtime();

      MPI_Send(
         buffer,
         bytes,
         MPI_BYTE,
         1,
         0,
         MPI_COMM_WORLD
      );

      double end = MPI_Wtime();

      printf(
         "Message Size = %d bytes, Communication Time = %.9f seconds\n",
         bytes,
         end - start
      );
   }
   else if (rank == 1)
   {
      MPI_Recv(
         buffer,
         bytes,
         MPI_BYTE,
         0,
         0,
         MPI_COMM_WORLD,
```

```
            MPI_STATUS_IGNORE
        );
    }

    free(buffer);
  }

  MPI_Finalize();

  return 0;
}
```

Compilation

mpicc communication_time.c -o communication_time

Execution

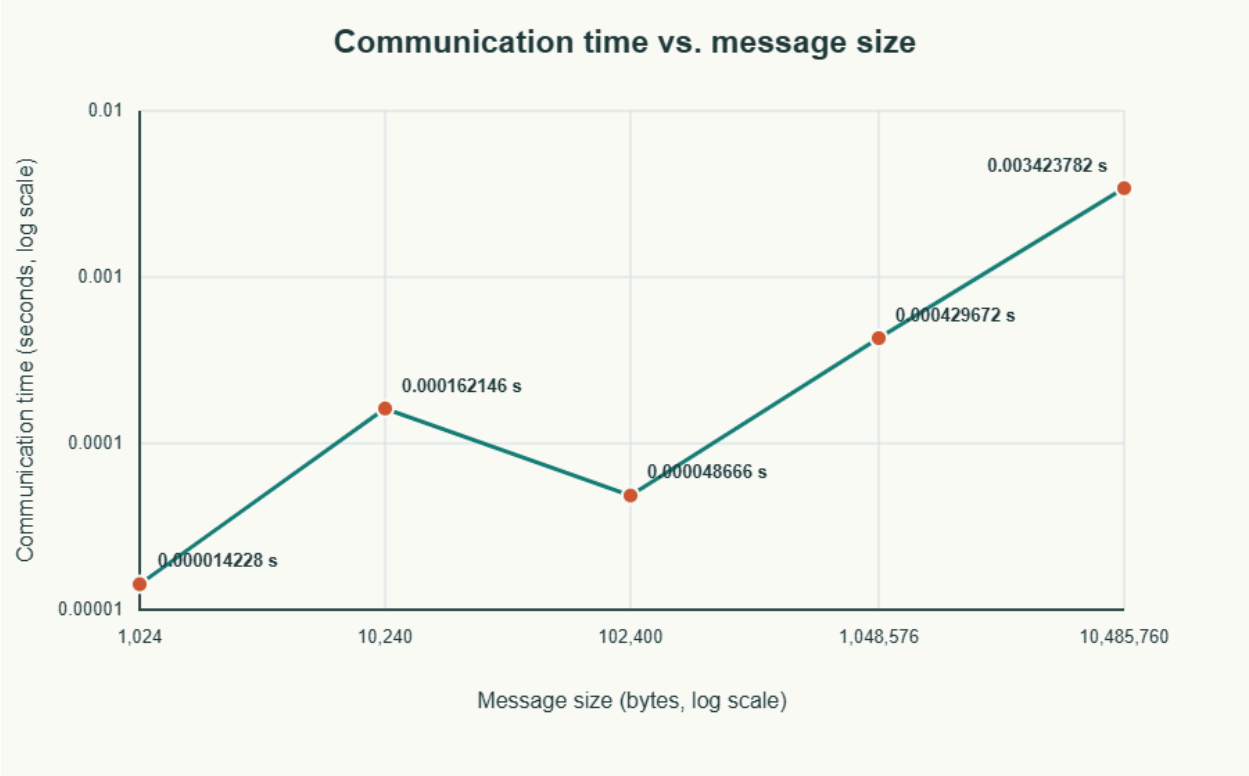
mpirun -np 2 ./communication_time

Results Table

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/Performance_Activity$ mpicc
 communication_time.c -o communication_time
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/Performance_Activity$ mpiru
n -np 2 ./communication_time
Message Size = 1024 bytes, Communication Time = 0.000026907 seconds
Message Size = 10240 bytes, Communication Time = 0.000197812 seconds
Message Size = 102400 bytes, Communication Time = 0.000036845 seconds
Message Size = 1048576 bytes, Communication Time = 0.000468715 seconds
Message Size = 10485760 bytes, Communication Time = 0.003400678 seconds
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/Performance_Activity$
```

Observation

The communication time generally increases as the message size increases. Small messages are more strongly affected by communication latency, while larger messages require more data-transfer time and network bandwidth.



Analyze

1. How does communication time change with message size?

As **message size increases, communication time generally increases** because more data has to be transferred between processes.

Communication time can be approximately represented as:

$$T_{comm} = T_{latency} + \frac{Message\ Size}{Bandwidth}$$

- **Small messages:** Latency (fixed communication setup time) has a larger effect.
- **Large messages:** The actual data-transfer time becomes more significant.

Therefore, sending a very large amount of data generally takes more time than sending a small amount of data.

2. Why is communication overhead important in distributed-memory computing?

In distributed-memory computing, processes have separate memories, so they need to communicate whenever they require data from another process.

Communication overhead includes the time spent on:

- Sending and receiving data
- Network data transfer
- Synchronization between processes
- Waiting for other processes

If communication overhead is high, processes may spend more time **communicating and waiting** than performing actual computation. This reduces the performance improvement gained from parallel processing.

Therefore, efficient distributed algorithms try to **minimize unnecessary communication**.

3. Why might MPI communication become a bottleneck?

MPI communication can become a **bottleneck** when processes spend a significant amount of time exchanging data instead of performing computation.

This can happen due to:

- **Large amounts of data** being transferred
- **Too many messages**, especially many small messages
- **Network latency or limited bandwidth**
- Frequent **synchronization** between processes
- **Load imbalance**, where some processes finish early and wait for others
- Increasing the number of processes without reducing the communication required

For example:

Ideal:
Computation → Computation → Computation

Poor communication pattern:
Compute → Wait → Communicate → Wait → Compute

If communication and waiting take longer than the useful computation, adding more MPI processes may provide little or no performance benefit.

> **In short:** MPI communication becomes a bottleneck when the cost of exchanging and synchronizing data becomes a significant part of the total execution time.

Challenge: Communication
Implement a communication between Process 0 and Process 1.

Requirement:
Process 0 sends a message to Process 1.
Process 1 sends the message back.
This continues for a specified number of iterations.
Process 0 Process 1
Send ---------->
<---------- Receive
Send ---------->
<---------- Receive
...
Measure the total communication time.

Calculate:
Round-Trip Time
RTT **=**
Total Time
Number of Iterations

Investigate how RTT changes with message size.

Objective

To measure the round-trip communication time between Process 0 and Process 1.

In each iteration:

Process 0 Process 1

Send ---------->

<---------- Send

The message travels from Process 0 to Process 1 and then back to Process 0.

Program: `ping_pong.c`

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank;
   int iterations = 1000;

   int sizes[] = {
      1024,
      10 * 1024,
      100 * 1024,
      1024 * 1024,
      10 * 1024 * 1024
   };

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);

   for (int s = 0; s < 5; s++)
   {
      int bytes = sizes[s];

      char *buffer = (char *)malloc(bytes);

      MPI_Barrier(MPI_COMM_WORLD);

      double start = 0.0;
      double end = 0.0;

      if (rank == 0)
      {
         start = MPI_Wtime();

         for (int i = 0; i < iterations; i++)
         {
            MPI_Send(
               buffer,
               bytes,
               MPI_BYTE,
               1,
               0,
```

```
            MPI_COMM_WORLD
        );

        MPI_Recv(
            buffer,
            bytes,
            MPI_BYTE,
            1,
            0,
            MPI_COMM_WORLD,
            MPI_STATUS_IGNORE
        );
    }

    end = MPI_Wtime();

    double total_time = end - start;
    double average_rtt = total_time / iterations;

    printf(
        "Size = %d bytes | Iterations = %d | Total Time = %.6f s | Average RTT = %.9f s\n",
        bytes,
        iterations,
        total_time,
        average_rtt
    );
}
else if (rank == 1)
{
    for (int i = 0; i < iterations; i++)
    {
        MPI_Recv(
            buffer,
            bytes,
            MPI_BYTE,
            0,
            0,
            MPI_COMM_WORLD,
            MPI_STATUS_IGNORE
        );

        MPI_Send(
```

```
            buffer,
            bytes,
            MPI_BYTE,
            0,
            0,
            MPI_COMM_WORLD
          );
        }
      }

      free(buffer);
    }

    MPI_Finalize();

    return 0;
}
```

Compilation

mpicc ping_pong.c -o ping_pong

Execution

mpirun -np 2 ./ping_pong

RTT Formula

The average round-trip time is calculated as:

[
RTT = \frac{Total\ Time}{Number\ of\ Iterations}
]

Performance Table

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/Challenge$ mpicc ping_pong.
c -o ping_pong
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/Challenge$ mpirun -np 2 ./p
ing_pong
Size = 1024 bytes | Iterations = 1000 | Total Time = 0.002259 s | Average RTT = 0.0000022
59 s
Size = 10240 bytes | Iterations = 1000 | Total Time = 0.006845 s | Average RTT = 0.000006
845 s
Size = 102400 bytes | Iterations = 1000 | Total Time = 0.032531 s | Average RTT = 0.00003
2531 s
Size = 1048576 bytes | Iterations = 1000 | Total Time = 0.167185 s | Average RTT = 0.0001
67185 s
Size = 10485760 bytes | Iterations = 1000 | Total Time = 1.575362 s | Average RTT = 0.001
575362 s
```

Observed Trend

The RTT generally increases as message size increases because every round trip requires the message to travel from Process 0 to Process 1 and back.

For small messages, latency has a relatively larger effect. As the message becomes larger, the time required to transfer the actual data becomes increasingly significant.

Advanced Challenge: Avoiding Communication Problems
Modify the program to investigate the behavior of blocking communication.

Consider:
MPI_Send()
MPI_Recv()

Experiment 1: Potentially Blocking Communication

The following pattern demonstrates why communication ordering must be considered carefully.

Problematic Pattern

```
// Process 0
MPI_Send(
  data,
  1,
  MPI_INT,
  1,
  0,
  MPI_COMM_WORLD
```

```
);

MPI_Recv(
   data,
   1,
   MPI_INT,
   1,
   0,
   MPI_COMM_WORLD,
   MPI_STATUS_IGNORE
);
// Process 1
MPI_Send(
   data,
   1,
   MPI_INT,
   0,
   0,
   MPI_COMM_WORLD
);

MPI_Recv(
   data,
   1,
   MPI_INT,
   0,
   0,
   MPI_COMM_WORLD,
   MPI_STATUS_IGNORE
);
```

Both processes attempt to send first.

If the MPI implementation cannot complete the send until the corresponding receive is posted, both processes can remain blocked at `MPI_Send()`.

```
Process 0                    Process 1

MPI_Send() ---------------------->

          <------------------ MPI_Send()

     WAIT                    WAIT
```

X X

Neither process reaches its `MPI_Recv()`, producing a deadlock.

Important Note

The behavior of `MPI_Send()` can depend on the MPI implementation and message size. Some implementations may buffer certain messages, so a small-message test may appear to work even though the communication pattern is unsafe.

Therefore, an intentionally deadlocking program should **not be left running indefinitely on a shared HPC system**.

Experiment 2: Safe Communication Using MPI_Sendrecv()

A safer way for two processes to exchange data is to use `MPI_Sendrecv()`.

Program: `sendrecv.c`

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank;
   int send_data;
   int recv_data;
   MPI_Status status;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);

   if (rank == 0)
   {
      send_data = 100;

      MPI_Sendrecv(
         &send_data,
```

```
        1,
        MPI_INT,
        1,
        0,
        &recv_data,
        1,
        MPI_INT,
        1,
        0,
        MPI_COMM_WORLD,
        &status
    );

    printf(
        "Process 0 sent %d and received %d\n",
        send_data,
        recv_data
    );
}
else if (rank == 1)
{
    send_data = 200;

    MPI_Sendrecv(
        &send_data,
        1,
        MPI_INT,
        0,
        0,
        &recv_data,
        1,
        MPI_INT,
        0,
        0,
        MPI_COMM_WORLD,
        &status
    );

    printf(
        "Process 1 sent %d and received %d\n",
        send_data,
        recv_data
```

```
    );
  }

  MPI_Finalize();

  return 0;
}
```

Compilation

mpicc sendrecv.c -o sendrecv

Execution

mpirun -np 2 ./sendrecv

Expected Output

Process 0 sent 100 and received 200
Process 1 sent 200 and received 100

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/Advanced_Challenge$ mpicc s
end_recv.c -o send_recv
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/A6/Advanced_Challenge$ mpirun
-np 2 ./send_recv
Process 0 sent 100 and received 200
Process 1 sent 200 and received 100
```

Observation

`MPI_Sendrecv()` allows each process to perform a send and receive as part of one coordinated operation. It is useful for exchange patterns and helps avoid the problematic ordering of two independent blocking sends.

Alternative Safe Solution: Non-Blocking Communication

MPI also provides non-blocking operations such as:

MPI_Isend();
MPI_Irecv();
MPI_Wait();

A process can initiate communication and continue with other operations before waiting for the communication to complete.

A simplified pattern is:

```
MPI_Irecv(
   &recv_data,
   1,
   MPI_INT,
   1,
   0,
   MPI_COMM_WORLD,
   &request
);

MPI_Isend(
   &send_data,
   1,
   MPI_INT,
   1,
   0,
   MPI_COMM_WORLD,
   &request
);

MPI_Wait(&request, MPI_STATUS_IGNORE);
```

For multiple simultaneous communications, separate `MPI_Request` variables should be used for each operation.

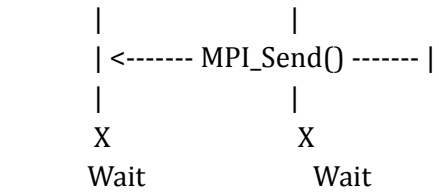
Study what happens if both processes attempt to send data before receiving.

Explain:
The purpose of this experiment is to understand how **blocking MPI communication** behaves when two processes attempt to send data before receiving it. The experiment also demonstrates how incorrect communication ordering can lead to **deadlock** and how the communication pattern can be redesigned to avoid it.

• **Why deadlock can occur.**

Consider two MPI processes:

```
Process 0                Process 1
   |                        |
   | -------- MPI_Send() ------> |
```



If both processes execute `MPI_Send()` first and the send operations cannot complete until the corresponding receive is posted, both processes can become blocked.

- Process 0 waits for Process 1 to receive its message.
- Process 1 waits for Process 0 to receive its message.
- Neither process reaches its `MPI_Recv()`.

This creates a **deadlock**, where both processes remain waiting indefinitely.

The exact behavior of `MPI_Send()` can depend on the MPI implementation and message size because an MPI implementation may buffer some messages. Therefore, a program that appears to work with a small message is not necessarily free from a communication-ordering problem.

• How message ordering affects communication.

MPI messages are matched using information such as:

- **Source rank**
- **Destination rank**
- **Message tag**
- **Communicator**

The order of communication operations therefore matters.

For example, suppose Process 0 sends:

Message A → Tag 10
Message B → Tag 20

and Process 1 expects them in the corresponding order:

Receive Tag 10
Receive Tag 20

The communication is straightforward.

However, if processes are designed with incompatible ordering, one process may wait for a message that the other process will not send until it receives something first. This can result in deadlock.

Thus, MPI programs should have a **well-defined communication sequence**.

• How communication can be redesigned to avoid deadlock. Do not deliberately run an unbounded deadlocking program on a shared HPC system. Use a controlled small test and terminate it safely if required.

One simple solution is to ensure that **one process receives before sending**, while the other process sends first.

For example:

Process 0 Process 1

MPI_Send() ----------->
MPI_Recv()
MPI_Recv() <-----------
MPI_Send()

A better general approach is to use **non-blocking communication**, such as:

MPI_Isend()
MPI_Irecv()

These allow the program to initiate communication without immediately blocking and then wait for completion using functions such as `MPI_Wait()`.

MPI also provides `MPI_Sendrecv()`, which can be particularly useful when two processes need to exchange data:

MPI_Sendrecv(...);

This combines a send and receive into a single operation and can avoid the problematic ordering of separate blocking sends.

Controlled Experiment

For a safe experiment, use only **2 MPI processes** and a small message.

A problematic pattern is:

```
// Process 0
MPI_Send(data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
MPI_Recv(data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &status);
```

and:

```
// Process 1
MPI_Send(data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
MPI_Recv(data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
```

Depending on the MPI implementation and message buffering, this may complete or may block. **Do not use an unbounded run on a shared HPC system.** If testing the potentially blocking version, use a short timeout or run it locally in a controlled environment and terminate it if it does not progress.

A safe redesigned version can use `MPI_Sendrecv()`:

```
MPI_Sendrecv(
   send_data, 1, MPI_INT, 1, 0,
   recv_data, 1, MPI_INT, 1, 0,
   MPI_COMM_WORLD, &status
);
```

For two processes, each process can exchange data with the other without requiring both processes to independently perform a blocking `MPI_Send()` first.

Conclusion

Blocking communication requires careful ordering of `MPI_Send()` and `MPI_Recv()`. If processes wait for one another in an incompatible order, **deadlock can occur**. Message tags, source/destination ranks, and communicators determine how messages are matched. Deadlocks can be avoided by designing a proper communication sequence or by using mechanisms such as **`MPI_Sendrecv()` or non-blocking `MPI_Isend()`/`MPI_Irecv()`**.

HPC / AI Case Study

Choose one application:
- **Weather forecasting**
- **Computational fluid dynamics**
- **Molecular dynamics**
- **Distributed deep learning**
- **Large-scale scientific simulation**
- **Graph processing**
- **Image processing**

HPC / AI Case Study: Image Processing Using MPI

Introduction

Image processing involves applying computational operations to images for tasks such as **image enhancement, filtering, edge detection, segmentation, and feature extraction**. Processing a large number of high-resolution images can require significant computational resources. Distributed computing can divide this workload among multiple processes and reduce the overall processing time.

1. Why is distributed computing required?

Large images contain millions of pixels, and processing a large collection of high-resolution images can be computationally expensive. A single processor may take considerable time to process them sequentially. Distributed computing allows the workload to be divided among multiple MPI processes, so several parts of the image or multiple images can be processed simultaneously.

2. What data needs to be exchanged between processes?

The data exchanged depends on the image-processing operation. Processes may need to exchange **image portions, pixel values, intermediate results, or boundary pixels**. For operations such as convolution and edge detection, a process may need pixels from neighboring regions to correctly process pixels near the boundary of its assigned region.

3. Which processes need to communicate?

Processes working on neighboring portions of an image may need to communicate with each other. For example, if an image is divided into horizontal sections, Process 1 may need boundary pixels from Process 0 and Process 2. A master process may also communicate with worker processes to distribute data and collect the final results.

4. How could MPI be used?

MPI can divide an image into smaller sections and assign each section to a different process. Functions such as `MPI_Send()` and `MPI_Recv()` can be used to exchange image data and boundary pixels between processes. After processing their assigned portions, processes can send their results back to a designated process, which can combine them to form the final processed image.

5. What would happen if communication time becomes larger than computation time?

If communication takes longer than the actual image processing, the benefits of parallelization decrease. Processes may spend more time **waiting for data or exchanging messages** than performing useful computation. This increases communication overhead and can result in poor performance or even make the distributed implementation slower than processing the image on a single system.

Conclusion

MPI can make large-scale image processing more efficient by distributing computation among multiple processes. However, effective **data distribution, communication management, and load balancing** are essential. The goal is to perform sufficient computation on each process while minimizing unnecessary communication so that the advantages of parallel processing are realized.

Questions

Basic
1. What is MPI?

MPI (Message Passing Interface) is a standard used for communication between processes in **parallel and distributed-memory computing**.

MPI provides functions that allow processes to:

- Send and receive data
- Communicate with other processes
- Synchronize their execution
- Perform collective operations such as broadcast, scatter, gather, and reduction

MPI is commonly used with **C, C++, and Fortran**.

2. What is an MPI process?

An **MPI process** is an independent instance of a program that is created when an MPI application is started.

For example:

mpirun -np 4 ./program

This starts **4 MPI processes**. Each process has its own memory space and executes the program independently.

The processes communicate with one another using MPI communication functions.

3. What is an MPI rank?

An **MPI rank** is a unique integer identifier assigned to each process **within a particular communicator**.

Ranks usually start from `0`.

For example, with 4 processes:

Process	Rank
Process 1	0
Process 2	1
Process 3	2
Process 4	3

A rank is used to identify the sender or receiver during communication.

For example:

MPI_Send(data, 10, MPI_INT, 2, 0, MPI_COMM_WORLD);

Here, `2` is the **rank of the destination process**.

4. What is the purpose of MPI_COMM_WORLD?

`MPI_COMM_WORLD` is the **default communicator provided by MPI**.

It contains **all MPI processes** that were started for the application.

It allows processes to communicate with each other and provides the context in which their ranks are interpreted.

Two commonly used functions are:

MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

- `MPI_Comm_size()` → gives the total number of processes.
- `MPI_Comm_rank()` → gives the rank of the current process.

5. What is the difference between MPI_Send() and MPI_Recv()?

`MPI_Send()`	`MPI_Recv()`
Sends data	Receives data
Used by the sending process	Used by the receiving process

Specifies the destination rank	Specifies the source rank
Transfers data from one process to another	Obtains data sent by another process

Example:

MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);

This sends `data` to **rank 1**.

The receiving process can use:

MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);

This receives data from **rank 0**.

Moderate
6. Differentiate between an MPI group and communicator.

An **MPI group** and an **MPI communicator** are related but serve different purposes.

MPI Group	MPI Communicator
Represents a collection of MPI processes	Represents a communication context involving a group of processes
Defines **who belongs to the group**	Defines **who can communicate and in what communication context**
Does not itself provide a communication context	Provides the context required for MPI communication
Mainly used for organizing/manipulating process groups	Used directly in functions such as `MPI_Send()` and `MPI_Recv()`

A simple way to remember it:

> **Group = collection of processes**
> **Communicator = group + communication context**

For example, you can create a group containing only processes with ranks `0, 1, 2`, and then create a communicator for those processes.

7. Why can two processes have different ranks in different communicators?

An MPI rank is **not a permanent identity of a process**. A rank is assigned **relative to a particular communicator**.

Consider:

MPI_COMM_WORLD

Rank 0
Rank 1
Rank 2
Rank 3

Suppose we create a new communicator containing only the original processes with ranks 1, 2, and 3:

New Communicator

New Rank 0 → Original Rank 1
New Rank 1 → Original Rank 2
New Rank 2 → Original Rank 3

Therefore, the same process could be:

Rank 2 in MPI_COMM_WORLD
Rank 1 in another communicator

This is useful because different groups of processes can communicate independently.

8. What information is specified when using MPI_Send()?

A typical `MPI_Send()` call is:

MPI_Send(buffer, count, datatype, destination, tag, communicator);

It specifies:

1. **`buffer`** – Address of the data to send
2. **`count`** – Number of elements to send
3. **`datatype`** – Type of the data, such as `MPI_INT` or `MPI_DOUBLE`
4. **`destination`** – Rank of the receiving process
5. **`tag`** – Identifier used to distinguish different messages

6. **`communicator`** – Communicator through which the message is sent

Example:

MPI_Send(data, 10, MPI_INT, 2, 100, MPI_COMM_WORLD);

This means:

> Send 10 integers from `data` to rank 2, using message tag 100 within `MPI_COMM_WORLD`.

9. What information is specified when using MPI_Recv()?

A typical `MPI_Recv()` call is:

MPI_Recv(buffer, count, datatype, source, tag, communicator, &status);

It specifies:

1. **`buffer`** – Memory location where received data will be stored
2. **`count`** – Maximum number of elements that can be received
3. **`datatype`** – Expected type of the data
4. **`source`** – Rank of the sending process
5. **`tag`** – Tag used to identify the required message
6. **`communicator`** – Communicator used for communication
7. **`status`** – Stores information about the received message

For example:

MPI_Recv(data, 10, MPI_INT, 0, 100, MPI_COMM_WORLD, &status);

This means:

> Receive up to 10 integers from rank 0 with tag 100 through `MPI_COMM_WORLD`.

10. Why does the order of send and receive operations matter?

The order matters because MPI communication operations must **match correctly**.

For a receive operation to obtain a particular message, properties such as the following must match:

- Communicator
- Source rank

- Message tag
- Compatible datatype/count requirements

For example:

Process 0 Process 1

Send A ----------> Receive A
Send B ----------> Receive B

The receiving operations must be able to match the corresponding messages.

Incorrect ordering can cause:

- A process to wait for a message that has not been sent
- Messages to be received in an unintended order
- A **deadlock**, where processes wait indefinitely for each other

Therefore, the communication sequence should be designed carefully.

Advanced
11. What is communication overhead?

Communication overhead is the additional time and computational resources required for processes to **exchange data and coordinate with each other**, rather than performing useful computation.

In MPI, communication overhead can include:

- Time to send and receive messages
- Data transfer time
- Network latency
- Synchronization time
- Processing required to manage communication

For example:

Useful computation → 80 ms
Communication → 20 ms
Total time → 100 ms

Here, 20 ms represents communication-related overhead.

A distributed algorithm should therefore try to perform enough computation locally to make communication worthwhile.

12. Explain how message size affects MPI performance.

Message size has a significant effect on MPI performance.

For **small messages**, the main concern is often **latency**. Even if only a few bytes are transferred, the communication has a fixed setup cost.

For **large messages**, the amount of data being transferred becomes more important. Larger messages generally require more time and network bandwidth.

A simplified model is:

$$T_{communication} = T_{latency} + \frac{Message\ Size}{Bandwidth}$$

Therefore:

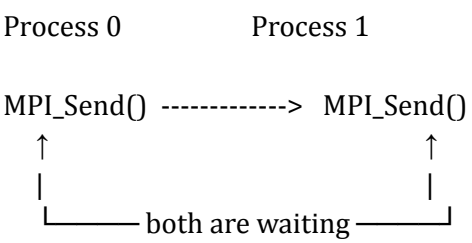
- **Small messages** → latency has a larger effect.
- **Large messages** → data-transfer time and bandwidth have a larger effect.

Sending many small messages can therefore be inefficient compared with combining data into fewer larger messages when the algorithm allows it.

13. What is deadlock in MPI?

A **deadlock** occurs when two or more processes wait indefinitely for an event or communication operation that cannot occur because each process is waiting for another process.

For example:



If both processes use blocking sends in a situation where neither send can complete until the other process receives the message, both processes can become stuck.

The program does not progress because each process is waiting for the other.

14. Why can blocking communication lead to deadlock?

Blocking communication means that a process may wait until the communication operation has progressed sufficiently or completed.

Consider:

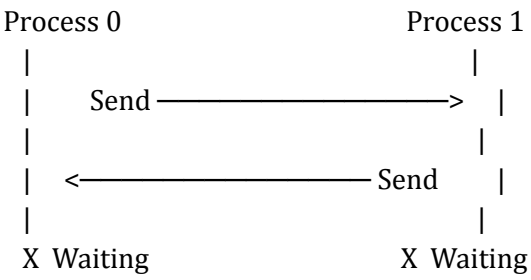
```
// Process 0
MPI_Send(data, ..., 1, ..., MPI_COMM_WORLD);
MPI_Recv(data, ..., 1, ..., MPI_COMM_WORLD, ...);
```

and:

```
// Process 1
MPI_Send(data, ..., 0, ..., MPI_COMM_WORLD);
MPI_Recv(data, ..., 0, ..., MPI_COMM_WORLD, ...);
```

Both processes attempt to **send first**.

If the MPI implementation cannot complete either send without the corresponding receive being posted, both processes wait:



Neither reaches its `MPI_Recv()`, resulting in **deadlock**.

Careful ordering, non-blocking operations, or appropriate MPI communication patterns can be used to avoid this problem.

15. Why is communication an important consideration when designing distributed-memory Algorithms?

Communication is important because processors in a distributed-memory system **do not share memory**. Whenever one process needs data from another process, that data must be communicated.

If an algorithm requires excessive communication, the communication overhead can become larger than the time saved through parallel processing.

For example:

Sequential:
Computation = 100 units

Parallel:
Computation per process = 25 units
Communication overhead = 30 units
Total = 55 units

The parallel version still provides a benefit here, but if communication becomes very large:

Computation per process = 25 units
Communication overhead = 100 units
Total = 125 units

parallel execution may become inefficient.

Therefore, distributed-memory algorithms should aim to:

- **Minimize unnecessary communication**
- **Reduce the amount of data transferred**
- **Avoid excessive synchronization**
- **Balance the workload among processes**
- **Perform more computation locally when possible**
- **Organize communication efficiently**

Performance Analysis
For the ping-pong experiment, prepare:
Message Size Iterations Total Time Average RTT
1 KB
10 KB
100 KB
1 MB
10 MB

Plot:
1. Message Size vs Communication Time
2. Message Size vs Round-Trip Time

Discuss the observed trend.

Answer the following:

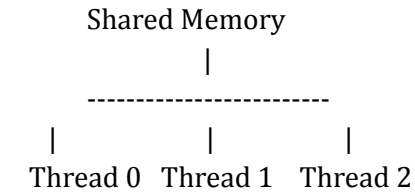
1. What is the fundamental difference between OpenMP and MPI?

The fundamental difference is **how they handle memory and communication**.

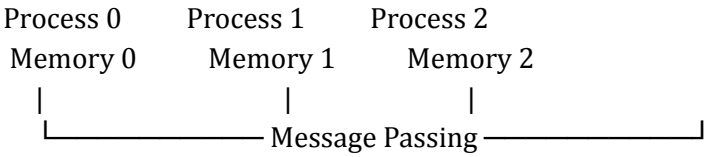
OpenMP	MPI
Primarily uses **shared-memory parallelism**	Primarily uses **distributed-memory parallelism**
Multiple threads share the same memory	Multiple processes have separate memory spaces
Threads communicate through shared variables	Processes communicate through message passing
Usually used within a single shared-memory machine	Can be used across multiple machines connected by a network
Parallelism is created using compiler directives such as `#pragma omp`	Parallelism and communication are managed using MPI functions

Example:

With OpenMP:



With MPI:



So, the simplest distinction is:

OpenMP → threads + shared memory
MPI → processes + separate memory + message passing

2. When would you choose MPI instead of OpenMP?

MPI would be chosen when the application needs to run across **multiple computers or nodes**, where each node has its own memory.

MPI is particularly suitable when:

- The problem is too large for the memory of a single machine.
- Computation needs to be distributed across multiple machines.
- Processes need explicit control over how data is distributed.
- The application is designed for a cluster or high-performance computing environment.
- Independent processes need to communicate using message passing.

For example, a very large matrix could be divided among several machines. Each machine performs calculations on its portion and uses MPI to exchange required data.

OpenMP is more suitable when the computation is primarily performed on **one shared-memory system** and its threads can efficiently work on the same data.

3. Why is data distribution important in MPI?

Data distribution is important because MPI processes have **separate memory spaces**. Each process needs to know which portion of the overall data it should work on.

Good data distribution helps to:

- Divide the workload among processes.
- Keep all processes busy.
- Reduce unnecessary communication.
- Avoid one process doing significantly more work than others.
- Improve overall parallel performance.

For example, suppose a matrix has 1000 rows and there are 4 MPI processes:

Process 0 → Rows 0–249
Process 1 → Rows 250–499
Process 2 → Rows 500–749
Process 3 → Rows 750–999

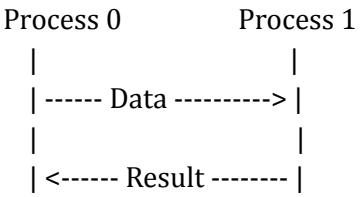
Each process can work on its own portion independently.

If the data is distributed poorly, some processes may finish quickly while others continue working. This creates **load imbalance** and reduces the benefit of parallelism.

4. What is the role of communication in distributed-memory applications?

Communication allows MPI processes to **exchange data and coordinate their work** because they cannot directly access each other's memory.

For example:



Communication is used for:

- Sharing required data
- Sending intermediate results
- Combining results from multiple processes
- Synchronizing processes
- Coordinating different stages of an algorithm

However, communication also introduces **overhead**. Therefore, an efficient distributed-memory application tries to minimize unnecessary communication while still providing the required data to each process.

5. What challenges arise when the number of MPI processes increases?

Increasing the number of MPI processes can provide more computational power, but it also introduces several challenges.

1. Communication overhead

More processes may need to exchange more messages.

Few processes → Less communication
Many processes → Potentially more communication

2. Load balancing

The work must be distributed evenly. If some processes have much more work than others, the faster processes may remain idle.

3. Synchronization

Processes may need to wait for one another before continuing. With more processes, synchronization can become more expensive.

4. Memory requirements

Each process may need its own copy of some data, increasing the overall memory requirement.

5. Deadlocks and communication errors

As communication patterns become more complicated, incorrect ordering of send/receive operations can lead to deadlocks.

6. Scalability

Adding more processes does not always result in proportional speedup. Eventually, communication and synchronization overhead can limit performance.

Therefore:

> **More MPI processes do not automatically mean better performance. The algorithm and its communication pattern must also scale efficiently.**

6. How could the concepts learned in this assignment be used in the next MPI assignment involving matrix computations?

The concepts learned in this assignment provide the **basic foundation for implementing parallel matrix computations using MPI** in the next assignment.

The concepts can be applied as follows:

- **MPI processes** can be used to divide the matrix computation among multiple processes.
- **MPI ranks** can identify which process is responsible for a particular portion of the matrix.
- `MPI_COMM_WORLD` can be used to establish communication among all participating processes.
- **Data distribution** can be used to divide matrix data among the processes.
- `MPI_Send()` **and** `MPI_Recv()` can be used when processes need to exchange matrix data or intermediate results.
- The understanding of **communication overhead** can help in designing the matrix computation efficiently.
- Knowledge of **deadlock and communication ordering** can help avoid communication problems when processes exchange data.

Thus, the concepts from this assignment will help in understanding how a matrix computation can be **divided among MPI processes, executed in parallel, and coordinated through message passing** in the next assignment.

Conclusion:

This assignment provided an understanding of the fundamental concepts of **MPI and distributed-memory parallel computing**. It covered MPI processes, ranks, communicators, `MPI_COMM_WORLD`, message passing, data distribution, communication overhead, and deadlocks. The differences between **OpenMP and MPI** were also studied, along with the challenges involved in increasing the number of MPI processes. These concepts provide the necessary foundation for developing and understanding **parallel applications using MPI** and will be useful for implementing more complex computations in subsequent assignments.

Deliverables

Submit:

1. MPI installation and environment details.
2. Source code for all completed tasks.
3. Execution outputs.
4. MPI rank/group/communicator observations.
5. Performance measurements.
6. Communication-time graphs.
7. Answers to discussion questions.
8. AI/HPC case study.
9. Brief conclusion.